arquitecturas web

bloque 2.1: fundamentos web

1. Cómo funciona la web

De Docker a arquitecturas web

- Con **Git** aprendimos a versionar código, y con **Docker** a empaquetarlo y ejecutarlo igual en cualquier sitio.
- Nos falta una pieza: ¿cómo "habla" exactamente una aplicación web con quien la usa, y cómo se organiza por dentro?
- Eso es lo que vemos en esta UD: **arquitecturas web**.

Arquitectura cliente-servidor

- **Cliente**: quien pide algo (normalmente, el navegador).
- **Servidor**: quien responde a esa petición (un programa escuchando en una máquina).
- Es el modelo base de toda la web: el cliente inicia la comunicación, el servidor contesta.

Un vistazo rápido al viaje de una petición

```
navegador → DNS (resuelve el nombre) → servidor (IP:puerto) → respuesta → navegador pinta la página
```

Vamos a desmontar cada pieza de este esquema.

HTTP: el idioma de la web

- **HTTP** (HyperText Transfer Protocol) es el protocolo que usan cliente y servidor para entenderse.
- Es un protocolo de **texto**, basado en el esquema **petición → respuesta**.
- **HTTPS** es HTTP cifrado con TLS: mismo protocolo, viaja protegido. Lo veremos en detalle en la UD3 (seguridad).

Métodos HTTP

- **GET**: pedir un recurso (no debería modificar nada en el servidor).
- **POST**: enviar datos para crear o procesar algo (p. ej. un formulario).
- **PUT**: reemplazar un recurso completo.
- **PATCH**: modificar parcialmente un recurso.
- **DELETE**: eliminar un recurso.

HTTP no es el único protocolo de aplicación

- HTTP es el más usado en la web, pero no el único protocolo construido sobre este mismo esquema petición-respuesta.
- **WebDAV** (Web Distributed Authoring and Versioning): extiende HTTP con métodos adicionales (`PROPFIND`, `MKCOL`, `COPY`, `LOCK`…) para gestionar ficheros y carpetas remotas como si fuera un sistema de ficheros (lo usan, p. ej., algunos clientes de sincronización de documentos).
- Otros protocolos de aplicación que convive con HTTP en una infraestructura típica: **FTP/SFTP** (transferencia de ficheros, UD4), **SMTP** (envío de correo), **SSH** (acceso remoto seguro).
- Todos comparten la idea de cliente-servidor, pero cada uno está pensado para una tarea distinta: no todo tiene que resolverse "metiéndolo por HTTP".

GET vs POST, en la práctica

	GET	POST
Datos van en	la URL (query string)	el cuerpo de la petición
Visible en el historial	Sí	No directamente
Longitud típica	Limitada	Prácticamente sin límite
Uso típico	Buscar, leer, filtrar	Enviar un formulario, subir un fichero

Códigos de estado HTTP

Cada respuesta HTTP incluye un código numérico de 3 cifras que resume el resultado:

- **1xx** — Informativo (raro verlo en el día a día).
- **2xx** — Éxito (`200 OK`, `201 Created`, `204 No Content`).
- **3xx** — Redirección (`301 Moved Permanently`, `302 Found`).
- **4xx** — Error del cliente (`400 Bad Request`, `401 Unauthorized`, `403 Forbidden`, `404 Not Found`).
- **5xx** — Error del servidor (`500 Internal Server Error`, `502 Bad Gateway`, `503 Service Unavailable`).

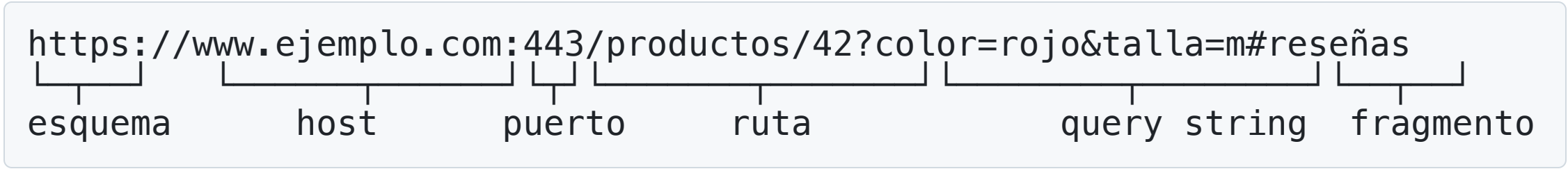
Códigos de estado que conviene distinguir bien

- `401 Unauthorized`: **no te has identificado** (o mal) — falta autenticación.
- `403 Forbidden`: te has identificado, pero **no tienes permiso** para eso — falta autorización.
- `404 Not Found`: el recurso no existe en esa URL.
- `500 Internal Server Error`: algo ha fallado **dentro del servidor**, no es culpa del cliente.

Cabeceras HTTP

- Pares `Clave: Valor` que acompañan a toda petición y respuesta, con metadatos.
- Petición: `Host`, `User-Agent`, `Accept`, `Cookie`, `Authorization`…
- Respuesta: `Content-Type`, `Content-Length`, `Set-Cookie`, `Cache-Control`…
- `Content-Type` le dice al cliente **cómo interpretar** el cuerpo de la respuesta (`text/html`, `application/json`, `image/png`…).

Anatomía de una URL



- **Esquema**: protocolo (`http` , `https`).
- **Host**: nombre de dominio o IP.
- **Puerto**: opcional, por defecto 80 (HTTP) o 443 (HTTPS).
- **Ruta**: qué recurso se pide.
- **Query string**: parámetros clave-valor.
- **Fragmento**: solo lo usa el navegador, nunca llega al servidor.

DNS: de nombres a direcciones IP

- Los ordenadores se localizan en red por su **dirección IP** (p. ej. `93.184.216.34`), no por nombres.
- **DNS** (Domain Name System) traduce nombres de dominio (`www.ejemplo.com`) a direcciones IP.
- Es, en esencia, la "agenda de contactos" de internet.

DNS, paso a paso (simplificado)

```
navegador → ¿tengo "www.ejemplo.com" en caché? no →
   pregunta a un servidor DNS (resolver) →
   resolver pregunta a servidores DNS jerárquicos (raíz → .com → ejemplo.com) →
   devuelve la IP → el navegador ya puede conectar por HTTP(S)
```

Por eso, al registrar un dominio nuevo, "tarda en propagarse": hay cachés DNS de por medio.

Repaso — Sección 1, Pregunta 1

¿Qué papel juega el DNS en una petición web?

A) Cifra la comunicación entre cliente y servidor
B) Traduce un nombre de dominio a una dirección IP
C) Comprime el contenido de la página antes de enviarla
D) Gestiona las cookies del navegador

Repaso — Sección 1, Pregunta 1 (solución)

✅ **B)** Traduce un nombre de dominio a una dirección IP

Repaso — Sección 1, Pregunta 2

Un cliente hace una petición a una URL que no existe en el servidor. ¿Qué código de estado HTTP es más probable recibir?

A) 200
B) 302
C) 404
D) 500

Repaso — Sección 1, Pregunta 2 (solución)

✅ **C)** 404

Repaso — Sección 1, Pregunta 3

¿Qué diferencia principal hay entre `401 Unauthorized` y `403 Forbidden`?

A) Son exactamente el mismo error, con distinto nombre

B) 401 indica que falta o falla la identificación; 403 indica que, aun identificado, no tienes permiso

C) 401 es un error del servidor y 403 del cliente

D) 403 solo aparece en peticiones POST

Repaso — Sección 1, Pregunta 3 (solución)

✅ **B)** 401 indica que falta o falla la identificación; 403 indica que, aun identificado, no tienes permiso

2. HTTP no recuerda nada: estado y sesiones

HTTP es "sin estado" (stateless)

- Cada petición HTTP es **independiente**: el servidor no recuerda, por sí mismo, tus peticiones anteriores.
- Problema: ¿cómo sabe un servidor que sigues siendo "tú" entre una petición y la siguiente (p. ej. que ya iniciaste sesión)?

Cookies: la memoria prestada del navegador

- El servidor manda una cookie con `Set-Cookie` en la respuesta.
- El navegador la guarda y la reenvía automáticamente (cabecera `Cookie`) en cada petición siguiente a ese dominio.
- Así, aunque HTTP no tenga estado, el servidor puede "reconocerte" mirando la cookie que llega.

Sesiones: el estado vive en el servidor

- Lo habitual no es guardar todos tus datos en la cookie, sino un **identificador de sesión** (un ID aleatorio).
- El servidor guarda el estado real (usuario, carrito, permisos…) asociado a ese ID, normalmente en memoria o en una base de datos.
- La cookie solo transporta la "llave" para encontrar ese estado.

El lado oscuro de las cookies

Las cookies no son solo comodidad: mal gestionadas, son una vía de ataque habitual.

- **Robo de sesión (session hijacking)**: si un atacante consigue leer tu cookie de sesión (p. ej. mediante un script malicioso inyectado en la página, un ataque **XSS**), puede hacerse pasar por ti sin conocer tu contraseña.
- **CSRF** (Cross-Site Request Forgery): el navegador envía tus cookies automáticamente a un sitio aunque la petición la haya iniciado otra página distinta, lo que un atacante puede aprovechar para ejecutar acciones en tu nombre sin que te des cuenta.
- **Seguimiento (tracking) entre sitios**: cookies de terceros usadas para rastrear tu navegación entre páginas distintas sin que seas plenamente consciente.

Mitigaciones básicas (se amplía en la UD3)

- `HttpOnly`: la cookie no puede leerse desde JavaScript, lo que reduce el riesgo de robo por XSS.
- `Secure`: la cookie solo se envía por HTTPS, nunca en claro por HTTP.
- `SameSite`: restringe que la cookie se envíe en peticiones iniciadas desde otros sitios, mitigando CSRF.
- Ninguna de estas opciones sustituye a validar bien la aplicación: son una capa extra de defensa, no la única.

Repaso — Sección 2, Pregunta 1

¿Por qué se dice que HTTP es un protocolo "sin estado" (stateless)?

A) Porque no puede transportar texto, solo binarios

B) Porque cada petición es independiente y el servidor no recuerda las anteriores por sí mismo

C) Porque no permite usar HTTPS

D) Porque no tiene códigos de estado

Repaso — Sección 2, Pregunta 1 (solución)

✅ **B)** Porque cada petición es independiente y el servidor no recuerda las anteriores por sí mismo

Repaso — Sección 2, Pregunta 2

¿Qué suele viajar realmente dentro de la cookie de sesión de un usuario ya autenticado?

A) La contraseña del usuario en texto plano
B) Todo el historial de navegación
C) Un identificador de sesión, usado para localizar el estado real guardado en el servidor
D) El código fuente de la página

Repaso — Sección 2, Pregunta 2 (solución)

✅ **C)** Un identificador de sesión, usado para localizar el estado real guardado en el servidor

Repaso — Sección 2, Pregunta 3

¿Qué cabecera usa el servidor para pedirle al navegador que guarde una cookie?

A) `Accept`
B) `Set-Cookie`
C) `Host`
D) `Content-Type`

Repaso — Sección 2, Pregunta 3 (solución)

✅ **B)** `Set-Cookie`

3. Arquitecturas en capas

El problema de meterlo todo junto

- Imagina una página escrita en un único fichero: HTML, consultas a la base de datos y lógica de negocio, todo mezclado.
- Funciona… hasta que hay que cambiar algo: tocar el diseño obliga a rozar la lógica de negocio y las consultas SQL.
- Solución: **separar responsabilidades en capas**.

Arquitectura de 1 capa

- Todo junto: interfaz, lógica y datos en el mismo bloque de código, ejecutándose en la misma máquina.
- Típico de scripts pequeños o aplicaciones de escritorio muy simples.
- No escala en complejidad ni en usuarios: cualquier cambio toca "todo a la vez".

Arquitectura de 2 capas (cliente-servidor clásica)

- Separa al menos **cliente** (interfaz) de **servidor** (lógica + datos).
- Ejemplo clásico: una aplicación de escritorio que se conecta directamente a una base de datos remota.
- Mejora el reparto, pero la lógica de negocio y el acceso a datos siguen mezclados en el mismo bloque.

Arquitectura de 3 capas

La más habitual hoy en aplicaciones web:

1. **Presentación**: lo que ve el usuario (HTML/CSS/JS, o una app móvil).
2. **Negocio** (o lógica de aplicación): reglas, validaciones, cálculos.
3. **Datos**: acceso y persistencia (base de datos, ficheros…).

Por qué separar en 3 capas

- Cada capa se puede **cambiar sin tocar las demás**: puedes rediseñar la interfaz sin tocar la base de datos.
- Cada capa se puede **escalar por separado**: más servidores de presentación si hay muchas visitas, sin duplicar la base de datos.
- Facilita el trabajo en equipo: unos desarrolladores en presentación, otros en negocio, otros en datos.

Arquitectura N-capas

- Generalización de la de 3 capas: se pueden añadir capas intermedias según haga falta.
- Ejemplo habitual: presentación → API/servicios → lógica de negocio → acceso a datos → base de datos.
- "3 capas" y "N capas" no son excluyentes: 3 capas es el caso particular más común de N capas.

Repaso — Sección 3, Pregunta 1

¿Cuál de estas NO es una de las tres capas típicas de una arquitectura en capas?

A) Presentación

B) Negocio

C) Datos

D) Virtualización

Repaso — Sección 3, Pregunta 1 (solución)

✅ **D)** Virtualización

Repaso — Sección 3, Pregunta 2

¿Cuál es la ventaja principal de separar la lógica de negocio de la capa de datos?

A) La aplicación consume menos electricidad
B) Se puede cambiar cómo se almacenan los datos sin reescribir las reglas de negocio, y viceversa
C) Hace que HTTP deje de ser stateless
D) Elimina por completo la necesidad de una base de datos

Repaso — Sección 3, Pregunta 2 (solución)

✅ **B)** Se puede cambiar cómo se almacenan los datos sin reescribir las reglas de negocio, y viceversa

Repaso — Sección 3, Pregunta 3

¿Qué relación hay entre "arquitectura de 3 capas" y "arquitectura N-capas"?

A) Son tecnologías completamente distintas e incompatibles

B) N-capas es un caso particular de 3 capas

C) 3 capas es el caso particular más habitual de la generalización N-capas

D) No tienen ninguna relación

Repaso — Sección 3, Pregunta 3 (solución)

✅ **C)** 3 capas es el caso particular más habitual de la generalización N-capas

4. Arquitecturas a gran escala: cómo se organizan los servicios

De las capas al tamaño del sistema completo

Las capas organizan **una** aplicación por dentro. Pero cuando el sistema crece (muchos equipos, mucho tráfico), hay que decidir también cómo se reparte **el sistema completo** en piezas desplegables. Ahí entran estos estilos de arquitectura.

Monolito

- Toda la aplicación (sus distintas funcionalidades) se construye, se despliega y se escala **como una sola unidad**.
- Internamente puede estar perfectamente organizada en capas: "monolito" no significa "código desordenado".
- Ventaja: sencillo de desarrollar y desplegar al principio. Inconveniente: a medida que crece, cualquier cambio exige volver a desplegar todo.

SOA (Service-Oriented Architecture)

- El sistema se divide en **servicios** que se comunican entre sí (típicamente mediante protocolos como SOAP o colas de mensajes).
- Precursora directa de los microservicios, pero con servicios normalmente más grandes y un middleware de comunicación centralizado (el "ESB", Enterprise Service Bus).
- Habitual en entornos corporativos grandes con sistemas heredados.

Microservicios

- El sistema se divide en **servicios pequeños e independientes**, cada uno con su propia responsabilidad, que se despliegan por separado.
- Cada microservicio puede tener su propia base de datos y, en principio, se puede escalar y desplegar sin tocar los demás.
- Encajan muy bien con Docker: cada microservicio suele vivir en su propio contenedor. **Profundizaremos en esto en la UD5 (CI/CD)**.

Monolito vs. microservicios

	Monolito	Microservicios
Despliegue	Una sola unidad	Independiente por servicio
Complejidad inicial	Baja	Alta (hay que coordinar servicios)
Escalado	Toda la app a la vez	Servicio por servicio
Punto de fallo	Uno afecta a todo	Un fallo puede quedar aislado
Equipo pequeño/proyecto simple	Buena opción	Puede ser sobreingeniería

Arquitectura orientada a eventos (Event-Driven)

- Los componentes no se llaman directamente entre sí: publican y reaccionan a **eventos** ("pedido creado", "pago confirmado") a través de un intermediario (cola o *broker* de mensajes).
- Desacopla emisor y receptor: el que publica el evento no necesita saber quién lo va a consumir.
- Muy usada junto con microservicios para que no dependan directamente unos de otros.

Serverless / FaaS

- **FaaS** (Function as a Service): despliegas funciones sueltas; la plataforma en la nube decide cuándo y dónde ejecutarlas.
- No significa que "no haya servidores": significa que **tú no los gestionas** (ni los enciendes, ni los parcheas, ni decides cuántos hay).
- Se suele pagar solo por el tiempo de ejecución real, no por tener el servidor encendido todo el día.

Peer-to-peer (P2P)

- No hay un servidor central claro: cada nodo actúa a la vez como cliente y como servidor.
- Ejemplo típico: redes de compartición de ficheros (BitTorrent).
- Poco habitual en aplicaciones web de negocio "normales", pero útil conocerlo como contraste frente al modelo cliente-servidor.

Repaso — Sección 4, Pregunta 1

¿Cuál es la diferencia principal entre un monolito y una arquitectura de microservicios?

A) El monolito nunca puede usar una base de datos

B) El monolito se despliega como una sola unidad; los microservicios se despliegan de forma independiente

C) Los microservicios no pueden escalar nunca

D) No hay ninguna diferencia real entre ambos

Repaso — Sección 4, Pregunta 1 (solución)

✅ **B)** El monolito se despliega como una sola unidad; los microservicios se despliegan de forma independiente

Repaso — Sección 4, Pregunta 2

En una arquitectura orientada a eventos, ¿cómo se comunican típicamente los componentes?

A) Llamándose directamente unos a otros por su dirección IP
B) Publicando y reaccionando a eventos a través de un intermediario (cola/broker), sin conocerse directamente
C) Compartiendo un único fichero de texto
D) No se comunican nunca entre sí

Repaso — Sección 4, Pregunta 2 (solución)

✅ **B)** Publicando y reaccionando a eventos a través de un intermediario (cola/broker), sin conocerse directamente

Repaso — Sección 4, Pregunta 3

¿Qué significa realmente "serverless" en el contexto de FaaS?

A) Que la aplicación no necesita ningún servidor físico en ningún lugar del mundo
B) Que el desarrollador no gestiona los servidores: la plataforma los aprovisiona y escala por él
C) Que solo funciona sin conexión a internet
D) Que es obligatorio usar microservicios

Repaso — Sección 4, Pregunta 3 (solución)

✅ **B)** Que el desarrollador no gestiona los servidores: la plataforma los aprovisiona y escala por él

5. Arquitecturas de diseño interno del código

De cómo se reparte el sistema a cómo se organiza el código

Los estilos de la sección anterior deciden cómo se reparte **el sistema** en piezas desplegables. Estos patrones deciden cómo se organiza **el código dentro de cada pieza** (cada capa de presentación, cada microservicio…).

MVC (Modelo-Vista-Controlador)

- **Modelo**: datos y lógica de negocio.
- **Vista**: lo que se muestra al usuario.
- **Controlador**: recibe la petición, coordina Modelo y Vista.
- Es, con diferencia, el patrón más usado en frameworks web (Spring MVC, Django, Ruby on Rails, Laravel...).

Variantes de MVC: MVP y MVVM

- **MVP** (Modelo-Vista-Presentador): el Presentador asume más lógica de la Vista; la Vista queda casi "tonta", lo que facilita probarla automáticamente.
- **MVVM** (Modelo-Vista-Vistamodelo): el ViewModel expone el estado de forma reactiva; la Vista se "sincroniza" sola con él. Muy habitual en frameworks frontend modernos (Vue, Angular).
- Los tres resuelven el mismo problema: separar qué se ve de cómo se calcula.

Arquitectura Hexagonal (puertos y adaptadores)

- La lógica de negocio ("el dominio") se coloca en el centro, totalmente aislada de detalles técnicos.
- Se comunica con el exterior a través de **puertos** (interfaces) y **adaptadores** (implementaciones concretas: una API REST, una base de datos, una cola de mensajes...).
- Ventaja clave: puedes cambiar la base de datos o el framework web **sin tocar** la lógica de negocio.

Clean Architecture / Onion Architecture

- Familia de ideas muy cercana a la Hexagonal: capas concéntricas, con las reglas de negocio en el centro y los detalles técnicos (frameworks, UI, base de datos) en los anillos exteriores.
- Regla común a las tres (Hexagonal, Clean, Onion): **las dependencias siempre apuntan hacia dentro**, nunca el dominio depende de un detalle técnico externo.

¿Por qué importan estos patrones?

- Facilitan las **pruebas automáticas**: puedes probar la lógica de negocio sin levantar una base de datos real.
- Facilitan el **mantenimiento a largo plazo**: cambiar un framework o una base de datos no debería obligar a reescribir las reglas de negocio.
- No son excluyentes entre sí con las capas o los microservicios: se pueden combinar (p. ej. un microservicio organizado internamente con Hexagonal y MVC en su capa de presentación).

Repaso — Sección 5, Pregunta 1

En el patrón MVC, ¿qué componente recibe la petición y coordina al resto?

A) El Modelo

B) La Vista

C) El Controlador

D) La base de datos

Repaso — Sección 5, Pregunta 1 (solución)

✅ **C)** El Controlador

Repaso — Sección 5, Pregunta 2

¿Cuál es la idea central de la arquitectura Hexagonal (puertos y adaptadores)?

A) Usar siempre una base de datos hexagonal

B) Aislar la lógica de negocio de los detalles técnicos, comunicándose con ellos mediante puertos y adaptadores

C) Sustituir completamente el patrón MVC

D) Es un sinónimo exacto de microservicios

Repaso — Sección 5, Pregunta 2 (solución)

✅ **B)** Aislar la lógica de negocio de los detalles técnicos, comunicándose con ellos mediante puertos y adaptadores

Repaso — Sección 5, Pregunta 3

¿Qué tienen en común la arquitectura Hexagonal, Clean Architecture y Onion Architecture?

A) Las tres exigen usar microservicios obligatoriamente

B) Las tres sitúan la lógica de negocio en el centro y hacen que las dependencias apunten siempre hacia dentro

C) Las tres son incompatibles con el patrón MVC

D) Ninguna de ellas se usa en aplicaciones web reales

Repaso — Sección 5, Pregunta 3 (solución)

✅ **B)** Las tres sitúan la lógica de negocio en el centro y hacen que las dependencias apunten siempre hacia dentro

6. Panorama general: ¿cuándo uso cada una?

Dos preguntas distintas, dos familias de arquitecturas

1. ¿Cómo reparto **el sistema completo** en piezas desplegables? → monolito, SOA, microservicios, event-driven, serverless, P2P.
2. ¿Cómo organizo **el código dentro de cada pieza**? → capas (1/2/3/N), MVC/MVP/MVVM, Hexagonal/Clean/Onion.

Estas dos familias **no compiten entre sí**: se combinan. Un microservicio puede estar organizado por dentro en 3 capas con Hexagonal.

Guía rápida de cuándo usar cada una

Escenario	Arquitectura razonable
Proyecto pequeño, equipo reducido, MVP	Monolito en 3 capas
Sistema grande, muchos equipos, despliegues independientes	Microservicios
Integrar sistemas heredados de una gran empresa	SOA
Picos de carga impredecibles, tareas puntuales	Serverless/FaaS
Componentes que deben reaccionar a cambios sin acoplarse	Event-driven
Lógica de negocio compleja que debe sobrevivir a cambios de framework	Hexagonal / Clean Architecture

El viaje completo de una petición, con todo lo visto hoy

```
navegador  --DNS-->  IP:puerto del servidor
        --petición HTTP (método, cabeceras, cookie de sesión)-->
capa de presentación (p. ej. Controlador de un MVC)
        --> capa de negocio (dominio, aislado si usa Hexagonal)
        --> capa de datos --> base de datos
        <-- respuesta HTTP (código de estado + cuerpo) <--
navegador pinta la página
```

Qué viene en el siguiente bloque

- Hoy nos hemos quedado en el **cómo se organiza y se reparte** una aplicación web.
- En el bloque **2.2** veremos **dónde vive físicamente** todo esto: servidores web vs. servidores de aplicaciones, contenido estático vs. dinámico, y qué significa realmente "desplegar".

Repaso — Sección 6, Pregunta 1

¿Qué relación hay entre "capas" (presentación/negocio/datos) y "microservicios"?

A) Son lo mismo con nombres distintos

B) Son incompatibles entre sí, nunca se usan juntos

C) Son dos decisiones de arquitectura distintas y complementarias: un microservicio puede, a su vez, estar organizado internamente en capas

D) Las capas sustituyen por completo a los microservicios

Repaso — Sección 6, Pregunta 1 (solución)

✅ **C)** Son dos decisiones de arquitectura distintas y complementarias: un microservicio puede, a su vez, estar organizado internamente en capas

Repaso — Sección 6, Pregunta 2

Para un sistema con picos de carga muy impredecibles y tareas puntuales de corta duración, ¿qué estilo de arquitectura suele encajar mejor?

A) Monolito de 1 capa
B) Serverless / FaaS
C) P2P
D) Ninguna arquitectura sirve para ese caso

Repaso — Sección 6, Pregunta 2 (solución)

✅ **B)** Serverless / FaaS

Repaso — Sección 6, Pregunta 3

¿Qué parte del "viaje completo de una petición" visto hoy corresponde al DNS?

A) Traducir el nombre de dominio a la IP del servidor, antes de poder enviar la petición HTTP
B) Guardar la sesión del usuario
C) Elegir qué arquitectura interna usa el servidor
D) Cifrar la respuesta del servidor

Repaso — Sección 6, Pregunta 3 (solución)

✅ **A)** Traducir el nombre de dominio a la IP del servidor, antes de poder enviar la petición HTTP

A practicar

Fundamentos web y arquitecturas